

Rebuilding a CampusEats service in REST

Goal In Assignment 3 you described a CampusEats service as a SOAP operation, a WSDL contract and a UDDI entry — all of it on paper. This time you will take a different service and build it: model it as resources, publish an OpenAPI contract, implement it in code, and make one cross-service call survive a bad network.

THE TASK

Choose one CampusEats service other than Payments. Write its `openapi.yaml` first, then implement at least four endpoints with correct status codes and a single error shape, make one endpoint safely retryable, harden one outbound call, and write up how the result compares with your Assignment 3 artefacts.

Work in the same repository as Tutorial 4, in a sibling folder. Keep your Tutorial 3 and Assignment 3 files — you will be quoting from them.

Part A · Model the service

#	Do this	How
A1	Pick your service	Choose Catalogue, Orders or Delivery. It must NOT be Payments, which was done in Tutorial 4. Use the boundary and data ownership you defined in Assignment 2; do not redesign them.
A2	List the operations you would have written as SOAP	Write three to five operations in the style of your Assignment 3 WSDL, e.g. <code>addItem(...)</code> , <code>getMenu(...)</code> , <code>setAvailability(...)</code> . This is your starting point, not your answer.
A3	Find the nouns	For each operation, ask what durable thing it creates or changes. Name that thing as a plural noun. Verbs such as <code>add</code> , <code>set</code> or <code>get</code> must not survive into any URL.
A4	Build the resource table	One row per endpoint, with five columns: method, URL, what it does, success code, failure codes. Minimum four rows, and at least one must be a sub-resource such as <code>/orders/42/cancellation</code> .
A5	Justify one hard choice	Pick the operation that mapped least comfortably onto a resource and write three or four sentences on how you resolved it and what you rejected.

Part B · Publish the contract

#	Do this	How
B1	Write <code>openapi.yaml</code> first	Before any handler code. Include <code>info</code> , <code>servers</code> , <code>paths</code> , and <code>components.schemas</code> . Every operation needs a <code>requestBody</code> or <code>parameters</code> , and at least two documented responses.
B2	Define your schemas once	Declare each request and response shape under <code>components.schemas</code> and refer to it with <code>\$ref</code> . Copy-pasted inline schemas lose marks.
B3	Declare every failure	Each operation must document its failure responses, not only the happy path. If your Assignment 3 WSDL declared faults, every one of them must appear here as a status code.
B4	Validate it	Run the file through <code>editor.swagger.io</code> or <code>openapi-spec-validator</code> . Include a screenshot or the terminal output showing zero errors.

Part C · Implement it

#	Do this	How
C1	Lay the service out	Follow the Tutorial 4 shape: <code>models.py</code> , <code>store.py</code> , <code>errors.py</code> , <code>app.py</code> , plus <code>tests/</code> . Storage may be an in-process dictionary. No other service may import your store.

#	Do this	How
C2	Separate record from representation	Your model class holds what you store; an <code>as_json()</code> method returns what you publish. They must differ in at least one field, and no credential or internal id may leak into a response.
C3	Implement at least four endpoints	One create, one read of a single item, one filtered list using a query string, and one state-changing sub-resource. All four must appear in <code>openapi.yaml</code> .
C4	Validate every request body	Write a <code>validate()</code> function and call it before touching the body's fields. This is the job your Assignment 3 XML Schema did for free — you are now doing it by hand.
C5	Return the right status codes	201 with a <code>Location</code> header on create. 200 on read. 202 if work continues after you reply. 400 for a malformed body, 404 for a missing resource, 409 for a state conflict, 422 for a valid request the domain refuses.
C6	Use one error shape everywhere	A single <code>problem()</code> helper producing <code>type</code> , <code>title</code> , <code>status</code> and <code>detail</code> on every failure. No endpoint may invent its own error format.
C7	Make one endpoint safely retryable	Pick the endpoint where a duplicate would do real damage. Support an <code>Idempotency-Key</code> header: store the key with the record, and on a repeat return the original result without redoing the work.
C8	Write four tests	One per behaviour: create succeeds with the right code and header; the idempotent repeat returns the original; one failure path returns the right 4xx; an unknown id returns 404. <code>pytest</code> must pass.

Part D · Make it survive the network

#	Do this	How
D1	Call one other service	Your service must make one real HTTP call to another CampusEats service — the Payments service from Tutorial 4 is fine. Resolve its address from an environment variable, never a hard-coded URL.
D2	Harden that call	Give it a timeout, and a retry with exponential backoff and jitter. Retry only what is safe to retry: a 4xx must never be retried, and any retried create must carry an idempotency key.
D3	Choose and document a fallback	Decide what your service does when the dependency is unreachable. Write two or three sentences in <code>NOTES.md</code> justifying the choice — including, if you chose to fail rather than degrade, why degrading would have been wrong.

WHAT TO SUBMIT

- ▶ The service folder: `openapi.yaml`, source files, and `tests/`, all running.
- ▶ `NOTES.md` containing your resource table from A4, your A5 justification, your D3 fallback reasoning, and the five answers below.
- ▶ A `curl` transcript, run with `-i`, showing at minimum:
 - a successful create, with its status code and `Location` header
 - the same request repeated with the same idempotency key, returning the original
 - a malformed body rejected, a missing resource, and a state conflict
- ▶ Terminal output or a screenshot of the OpenAPI validator reporting zero errors.
- ▶ Terminal output of `pytest` with all four tests passing.

ANSWER THESE IN NOTES.MD

1. Count the lines in your Assignment 3 WSDL and in your `openapi.yaml`. What is the difference actually made of? Name two things the WSDL declared that the OpenAPI file does not need to.
2. Quote one `soap:Fault` from your Assignment 3 work and show the status code and problem body that replaced it. Why is returning that error inside a 200 OK a problem for the network in between?

3. Which of UDDI's three moves — publish, find, bind — still exist in your new setup, and which disappeared? Explain what took over the job.
4. Your XML Schema was enforced before your code ran; your OpenAPI schema is not. Name the specific function in your code that now carries that responsibility, and one failure that would get through if you had not written it.
5. Name one part of your service where you would still choose the SOAP stack over REST, and state exactly what guarantee you would be buying. Answering “nowhere” needs a stronger argument than answering “somewhere”.

Submissions without the failure paths in the curl transcript will be returned unmarked. Name and Roll No of all team members on the first page.

Due before Lecture 6 · Dr. Primit Mazumdar · primit.mazumdar@iiitvadodara.ac.in